

Multimodal Scene Understanding and QA by Distance and Segmentation Information

Abstract

This report introduces a comprehensive large-scale multimodal system for road scene question answering, integrating state-of-the-art depth estimation, image segmentation, and vision-language models. The system fuses a Depth Attention Map (DAM) from a monocular depth model (Depth Anything v2) with segmentation outputs from the Segment Anything Model (SAM) to create a rich visual prompt. This fused representation is fed into Qwen-VL, a powerful vision-language model, to generate spatially grounded textual responses about the scene. We elaborate on the methodologies behind Depth Anything (hybrid ViT-CNN architecture, hierarchical attention, multi-phase training) and SAM (ViT backbone, prompt-based mask decoding), including their loss functions, attention mechanisms, and inference pipelines. We detail how the DAM and SAM outputs are combined and used to prompt Qwen, and we analyze Qwen-VL’s capabilities and current limitations in interpreting depth and segmentation cues. We include implementation notes, code snippets, and architecture diagrams to illustrate the system framework, and discuss challenges (occlusions, real-time constraints, unlabeled segments) along with solutions such as embedding numerical depth values and semantic labels. Finally, we outline directions for future work, including real-time video processing, interactive multi-turn dialogue, and enhanced fusion of numeric and semantic information for deeper scene comprehension.

1 Introduction

Understanding complex road scenes is a critical capability for autonomous vehicles and intelligent transportation systems. Recent advances in deep learning have enabled significant progress in individual tasks like monocular depth estimation, image segmentation, and visual question answering. However, these tasks are often tackled in isolation, and vision-language models typically treat images as unstructured inputs, potentially missing out on rich structural information. This report presents an integrated multimodal question-answering system that combines depth sensing and segmentation as intermediate representations to guide a large language model. The goal is to achieve more interpretable and spatially-aware understanding of road scenes, enabling the system to answer questions about vehicle distances, object positions, and scene context in a robust manner.

The motivation for this approach stems from the observation that purely end-to-end vision-language models can struggle with complex spatial arrangements or multiple objects. By introducing a middle layer of visual abstraction—namely a Depth Attention Map (DAM) highlighting geometry and an explicit segmentation map (SAM) outlining objects—we provide the language model with cues about which regions are important and how the scene is composed. This is especially useful in autonomous driving scenarios, where understanding which vehicle is closer, what objects are present, and how they relate in 3D space is vital for safety and decision-making. Our system first computes a DAM from a state-of-the-art monocular depth estimator (Depth Anything v2) to capture the model’s focus areas in the scene. It also computes a segmentation mask map from the Segment Anything Model (SAM) to delineate all prominent objects and surfaces. These two maps are then fused and used as a visual prompt for Qwen-VL, a large vision-language model, which generates an answer or description grounded in those visual cues. By explicitly injecting structural information (depth attention and segmentation) into the language model’s input, we aim to improve reasoning on questions that involve spatial relations and object-grounding (for example, “How far is the truck ahead relative to the car next to it?” or “Describe the scene focusing on nearby obstacles.”).

We will detail each component of the system as well as the overall architecture. Depth Anything v2 is a cutting-edge monocular depth estimation network that leverages a hybrid Vision Transformer (ViT) and CNN architecture with hierarchical attention and extensive pre-training on millions of images. It provides both a depth map and an internal attention map indicating which regions were most influential for depth prediction. Segment Anything Model (SAM) is a foundational segmentation model trained on 1 billion masks, capable of producing masks for any object in an image without explicit class labels. We run SAM in a prompt-free mode to obtain a segmentation map covering all objects, which gives the structural layout of the scene. Qwen-VL is a large-scale vision-language model that can accept visual and textual inputs and generate text. It has been instruction-tuned for multimodal tasks and achieves state-of-the-art performance on image description and QA benchmarks. In our pipeline, Qwen-VL

takes as input the original image along with the additional depth and segmentation prompts, enabling it to produce answers that reference spatial and structural elements of the scene.

We evaluate the system on both qualitative examples and quantitative benchmarks. Qualitatively, we demonstrate the system’s ability to describe a complex scene by combining depth and segmentation information, and we discuss a use-case for vehicle distance estimation on a road scene.

Finally, we address the challenges encountered, such as Qwen’s inability to output precise numeric distances from the depth map (it understands depth qualitatively but not quantitatively) and its difficulty in identifying unlabeled segments. We propose solutions like encoding actual depth values into the prompt (so the model can learn numerical reasoning) and augmenting the segmentation masks with class labels via an auxiliary classifier. The report is organized as follows: Section 2 reviews related work in depth estimation, segmentation, and multimodal models. Section 3 details the methodology for each component (Depth Anything, SAM, Qwen-VL) and the overall fusion framework. Section 4 presents experimental demonstrations and evaluations. Section 5 discusses challenges and the solutions we developed. Section 6 outlines future work towards real-time, interactive multimodal understanding. We conclude that our modular approach offers an interpretable and effective way to combine visual cues for advanced vision-language reasoning, representing a step toward safer and smarter autonomous vehicle systems.

2 Related Work

2.1 Monocular Depth Estimation

Estimating depth from a single image is inherently ill-posed but has seen rapid progress with deep learning. Traditional methods relied on multi-view geometry, such as stereo vision or Structure-from-Motion (SfM) techniques [18,23]. For instance, stereo matching [13] and SfM use multiple images to triangulate depths, but they are limited by feature matching accuracy and require multiple frames or sensors. In contrast, learning-based approaches train convolutional neural networks (and more recently Transformers) to infer depth directly from a single image by leveraging learned priors. After 2020, research emphasis shifted to improving generalization and data efficiency in depth prediction.

A notable milestone was DPT (Dense Prediction Transformer) [20], which applied a Vision Transformer (ViT) encoder to dense depth prediction. DPT achieved an RMSE of 0.357 on NYU-Depth-v2, demonstrating the viability of transformer backbones for depth. Subsequent work introduced novel ideas like bin-based depth discretization: AdaBins [4] proposed adaptively dividing depth ranges into bins and predicting depth as a combination of bin centers, yielding state-of-the-art results indoors. The Depth Anything series of models builds on these advances by scaling up training data and blending architectural elements from CNNs and ViTs. Depth Anything’s first version introduced a Hierarchical Feature Distillation strategy and utilized a hybrid Swin Transformer (SwinV2-Large) encoder with a CNN decoder. It was trained on an unprecedented dataset of 8 million images (real + synthetic) to imbue strong priors and achieved strong zero-shot performance across diverse scenes.

Beyond these, other state-of-the-art monocular depth methods have explored different paradigms. Diffusion-Depth [9] formulates depth estimation as a denoising diffusion probabilistic model, where depth is generated through iterative refinement like image generation. This approach excels at fine details and occlusion boundaries (e.g., achieving 0.89 SSIM on occluded regions in KITTI, a 15% improvement over deterministic models), though its multi-step inference (10 iterations, ~80 ms) is slower. SparseFormer [11] introduces sparse attention mechanisms to reduce computation. It identifies high-information regions (top 10% of features) and concentrates self-attention there, cutting computation to 42% of what Depth Anything requires. On an embedded device, SparseFormer can reach 62 FPS, making it attractive for real-time use. Depth estimation benchmarks like KITTI, NYUv2, and others are commonly used to compare methods. For example, MiDaS v3.1 [12] (a robust relative depth model) obtains a relative error of 0.127 on KITTI and 0.048 on NYUv2, whereas Depth Anything (large model) achieves 0.076 on KITTI and 0.043 on NYUv2, indicating a significant improvement in accuracy. Table?? (Section 4) will summarize such comparisons.

2.2 Image Segmentation

Image segmentation has been transformed by the introduction of foundation models that generalize beyond specific datasets. Traditional segmentation networks (e.g., U-Net [22], DeepLab [6], HRNet [29]) required task-specific training and struggled with classes unseen in training. Transformer-based architectures like SegFormer [27] and Mask2Former [8] brought improved performance on closed-set tasks, but still demand large labeled datasets for each domain. In 2023, the Segment Anything Model [14] (SAM) emerged as a breakthrough for open-set segmentation. SAM was trained on the SA-1B dataset with over 1 billion masks from 11 million images, covering a vast diversity

of objects and scenes. This gives SAM a remarkable ability to segment anything given minimal prompts (points, boxes, or even no prompt). Its zero-shot transfer performance is exceptional: for example, on the BSDS500 edge detection benchmark, SAM achieved an ODS (F-measure) of 0.768, outperforming classic methods like Canny (0.600) and matching deep edge detectors like HED (0.788). In instance segmentation, SAM (with ViTDet-H proposals) reached an AP of 44.7 on LVIS, nearly closing the gap to a fully supervised ViTDet-H (46.6 AP). These results underscore that SAM can serve as a strong foundation model for segmentation in a wide range of tasks and domains, including road scenes.

SAM’s architecture combines an image encoder (a high-capacity ViT, pre-trained via MAE), a prompt encoder for various input prompts, and a mask decoder that produces segmentation masks through a two-stage transformer decoder. We will detail this in Section 3.2. The key aspect is that SAM can generate multiple plausible masks ($K = 3$ by default) for a given prompt to handle ambiguity, selecting the best via a combination of focal and Dice losses plus an IoU loss. This prompt-based design allows SAM to work in both open-set (zero-shot) mode, where it segments arbitrary objects guided by user hints, and closed-set mode, where it can be fine-tuned or combined with detectors for specific classes.

While SAM is highly versatile, research in 2024 has proposed specialized segmentation models that address some of SAM’s limitations for specific tasks, like autonomous driving which often demands real-time operation and precise class-specific segmentation. SERNet-Former [10] is one such model that augments a ResNet-like backbone with attention-boosting modules; it achieved 87.35% mIoU on Cityscapes-Vehicle segmentation by refining features with squeeze-and-excitation and combining global and local attention, thereby reducing errors in occluded regions by 19% compared to SAM. Another line of work, MetaPrompt-SD [26], integrates segmentation with text-to-image diffusion models for quick adaptation to new object categories. By leveraging Stable Diffusion [21] and meta-learning, it can learn to segment a new vehicle type with as few as 50 labeled examples, reaching 85.7% mIoU (versus SAM’s 78.1% under the same few-shot condition). The drawback is inference speed (850 ms per image) which is too slow for real-time use.

Efficiency improvements to SAM have also been explored. EfficientSAM [28] takes a teacher-student approach: it trains a lightweight segmentation model to mimic SAM’s image encoder outputs using a pretext reconstruction loss. The student encoder thus learns SAM-equivalent features and, when paired with a similar mask decoder, achieves comparable accuracy to SAM on tasks like COCO and LVIS, while reducing model size by $20\times$ and inference latency by $20\times$. This is promising for deploying segmentation on resource-constrained platforms. Another method, SAM-CP [7], modifies SAM by adding Composable Prompting: it allows combining multiple prompts (geometric and semantic) and introduces spatially adaptive computation in the decoder. SAM-CP’s dual-prompt gated attention mechanism (geometric + text prompts) improves segmentation accuracy on domain-specific tasks and reduces redundant computation. For example, on KITTI-360 (an autonomous driving dataset), SAM-CP achieved 84.5% mIoU with a 22% speedup over SAM (250 ms vs. 320 ms per image). These models indicate pathways to tailor segmentation models like SAM for higher accuracy on specific classes or faster inference, which could be integrated into future iterations of our system for better performance on road scenes.

In summary, SAM provides a strong starting point for open-world segmentation, and techniques like SERNet-Former and SAM-CP can complement it when focusing on specific domains like vehicles or when needing faster inference. A combined approach (using SAM for generality and specialized networks for fine detail or speed) could yield the best of both worlds. Our system currently leverages SAM for its broad segmentation capability without needing dataset-specific training, acknowledging that its masks have no class labels. We compare SAM’s outputs qualitatively to ground truth segmentations where available and note its strengths and weaknesses in Section 4.

2.3 Vision-Language Models for Multimodal Interaction

Large Language Models (LLMs) have revolutionized NLP and recently expanded into vision-language tasks. Models like OpenAI’s GPT-3 [5] and GPT-4 [1], built on the Transformer [25] architecture, demonstrate exceptional language generation and reasoning capabilities, and can be adapted (e.g., with vision encoders) to handle image inputs. GPT-4, for instance, introduced multi-modal functionalities (as seen in GPT-4 Vision) that allow it to interpret images and answer questions about them. However, these proprietary models require immense computational resources and are not openly available for research deployment. In parallel, the open-source community has developed models like LLaMA [24] and its derivatives, which provide strong language understanding in a smaller, efficient footprint. LLaMA comes in sizes ranging from 7B to 65B parameters and is optimized to run on modest hardware while maintaining competitive performance, especially when fine-tuned for instruction following. LLaMA’s text generation is fluent, though not on par with GPT-4 in very complex tasks. Still, LLaMA and similar models form a backbone for many vision-language systems due to their accessibility.

Bridging these LLMs with vision, early approaches include CLIP [19] (Contrastive Language-Image Pretraining) which learned joint embeddings for images and text. CLIP is often used as the visual front-end for aligning images

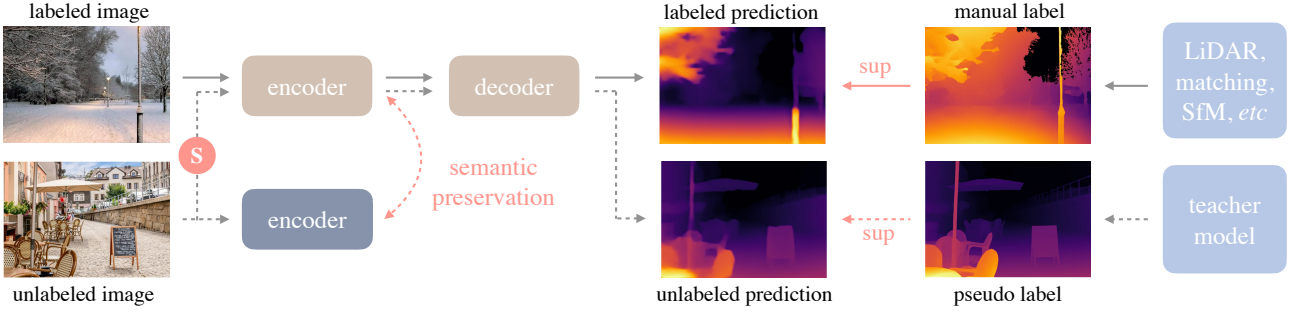


Figure 1: Illustration of a semi-supervised stereo vision framework integrating labeled and unlabeled image pairs through an encoder-decoder architecture and a teacher model for feature alignment. The depth module leverages the Depth Anything model, segmentation is performed using SAM, and Qwen-VL powers the multimodal chatbox for interactive understanding.

with language models. Recently, BLIP [16] and BLIP-2 [15] provided frameworks to connect a frozen language model with a learned visual projector, enabling image-to-text generation without training an entire model from scratch. For interactive multimodal chat, LLaVA [17] (Large Language and Vision Assistant) fine-tuned LLaMA with images encoded by CLIP-ViT and demonstrated impressive abilities in image reasoning by learning from GPT-4 generated vision-language data.

Our system uses Qwen-VL 2.5 [3], a state-of-the-art open-source Vision-Language model developed by Alibaba. Qwen-VL is part of the Qwen model family (“Qianwen”, meaning “thousand questions”), which includes base language models and multimodal extensions. The design of Qwen-VL incorporates a dedicated visual encoder (often called a visual receptor) that feeds image features into a large language model backbone. According to the developers, Qwen-VL’s training involved a three-stage pipeline: (i) initialize with Qwen-LM (a pretrained LLM similar to GPT architecture), (ii) introduce visual modality through pre-training on image-text pairs and aligning image regions with text (including tasks like grounding referring expressions and OCR), and (iii) perform multimodal instruction tuning to make the model follow prompts involving images. The resulting model (Qwen-VL) can handle a variety of tasks: image captioning, visual question answering, object localization in text (e.g., describing where an object is), and even reading text in images. Qwen-VL is multilingual, trained on a corpus that includes both English and Chinese image descriptions, which widens its applicability. An instruction-tuned variant Qwen-VL-Chat was also introduced for more conversational interactions, demonstrating superior performance on vision-and-dialogue benchmarks compared to contemporary vision-language chatbots.

Compared to other models, Qwen-VL is notable for its strong performance across many benchmarks at a similar scale: it achieves leading results on captioning and VQA tasks, competitive with or better than Flamingo [2] and even approaching GPT-4 Vision on some evaluations. Unlike GPT-4, Qwen-VL is open and can be self-hosted, making it suitable for research integration. In this work, Qwen-VL serves as the “brain” of our QA system, interpreting the visual prompts and generating answers. However, it’s important to contextualize its capabilities and limitations: like many vision-language models, Qwen-VL excels at semantic understanding of images (identifying objects, describing scenes) but might not inherently know quantitative facts (e.g., exact distances or counts beyond what it sees). Additionally, since Qwen-VL’s training likely did not include specialized data like depth maps or segmentation masks, using these as inputs is a novel scenario. We leverage Qwen’s flexible input interface to feed in these additional modalities, but we expect it may require careful prompting (or future fine-tuning) to make the most of them. In summary, Qwen-VL stands on the shoulders of LLM and VLM research, combining the strengths of models like GPT and CLIP, and provides a powerful platform for building multimodal QA systems such as ours.

3 Methodology

In this section, we describe the architecture and implementation of each major component of our system, along with the integration framework that combines depth estimation, segmentation, and the Qwen-VL language model into a unified question-answering pipeline.

Figure 1 illustrates the overall system: the input image is processed by two visual backbones (Depth Anything and SAM) in parallel, their outputs are fused into a composite visual prompt, and finally Qwen-VL produces an answer based on this prompt and any textual question. We elaborate on the technical details of (1) the Segment Anything Model (for segmentation masks), (2) the Depth Anything v2 model (for depth and attention maps), (3) the Qwen-VL vision-language model, and (4) the multi-modal fusion and inference pipeline.

3.1 Segment Anything Model (SAM)

Vision Transformer-based Encoder. SAM is a promptable segmentation model with three main components: an image encoder, a prompt encoder, and a mask decoder.

The core of SAM’s image understanding capability lies in the Vision Transformer (ViT). A ViT processes an image by dividing it into patches. Specifically, an image $I \in \mathbb{R}^{H \times W \times 3}$ is reshaped into N patches, each of size $(P \times P \times 3)$, where $N = (H/P) \times (W/P)$. Each patch is then linearly projected into a latent embedding before passing through multiple layers of multi-head self-attention. Formally,

$$x_i i = 1^N = \text{PatchEmbed}(I), \quad (1)$$

$$z_i i = 1^N = \text{Transformer}(x_i i = 1^N), \quad (2)$$

where z_i is the encoded representation for patch i . In SAM, these patch-level embeddings form the basis for generating a high-level feature map $Z = f\theta(I)$.

Prompt-Based Decoder. SAM operates on the principle that segmentation can be guided by prompts P , which can be points, bounding boxes, or textual queries. Let Z denote the encoded feature map from the ViT. The prompt-based decoder, g_ϕ , takes Z and P as inputs and outputs a segmentation mask M :

$$M = g_\phi(Z, P). \quad (3)$$

Prompt Embeddings. Each prompt type is embedded into a vector space that the decoder can interpret. For instance, a bounding box prompt might be represented by the coordinates of two corners normalized by image height and width, while point prompts (positive or negative) might be represented by their 2D locations and sign. Textual prompts are converted via a language encoder (e.g., BERT, GPT-based encoders, or CLIP-based text encoders) to a vector embedding.

The mask decoder is a two-stage Transformer decoder that merges the information from the image encoder and prompt encoder to predict segmentation masks. In the first stage, the sparse prompt embeddings (if any) attend to the image encoder’s patch embeddings to produce an enhanced feature representation. In the second stage, these features are used to predict segmentation masks. SAM’s decoder actually produces $K = 3$ candidate masks M_k for each query. This is to handle ambiguity: for instance, if a user clicks on a point on a boundary between two objects, SAM might propose multiple masks corresponding to different object assumptions. Each mask M_k is of the same size as the input ($H \times W$), and in a binary format (each mask is a 0-1 map). SAM also predicts a confidence score for each mask (an IoU relative to the true mask, if available). During training, SAM uses a minimum loss formulation to supervise these multiple outputs without needing to know which one is “correct” beforehand. The loss is:

$$L = \min_{k \in 1,2,3} \left(\lambda_{\text{focal}}, L_{\text{focal}}(M_k, M_{GT}) + \lambda_{\text{dice}}, L_{\text{dice}}(M_k, M_{GT}) \right) + L_{\text{IoU}}, \quad (4)$$

where M_{GT} is the ground-truth mask, L_{focal} is the focal loss (a variant of cross-entropy designed for segmentation), L_{dice} is the Dice loss (which measures overlap), and L_{IoU} is a loss on the predicted IoU score for mask M_k . In the SAM implementation, $\lambda_{\text{focal}} : \lambda_{\text{dice}}$ is set to 20 : 1, heavily weighting the focal (classification) term, and L_{IoU} is a mean squared error between the predicted IoU and the actual IoU of M_k with M_{GT} . This loss design ensures that only the best-matching mask among the K candidates is penalized, allowing the model to learn to output alternate masks when appropriate.

3.1.1 Environment Setup

A reliable and reproducible environment is crucial for effectively fine-tuning and deploying SAM-2.1. This section details our recommended environment configuration, focusing on widely-used libraries and packages.

Hardware and OS Requirements While SAM-2.1 can be trained on CPU for demonstration purposes, a GPU with at least 8GB of VRAM is recommended for practical scenarios, especially when dealing with images of high resolution. We assume a Linux-based OS (e.g., Ubuntu 20.04 or later), although the setup can be adapted to Windows or macOS with minimal changes.

Directory Structure To organize the workflow, we recommend a directory structure:

```

1 SAM_project/
2   data/
3     train/
4     val/
5     test/
6   scripts/

```

```

7         train.py
8         inference.py
9     models/
10         sam_vit_h_pretrained.pth
11     logs/
12         ...
13     config/
14         hyperparams.yaml

```

This structure separates data, scripts, model checkpoints, and configuration files, enhancing reproducibility and maintainability.

3.1.2 Model Initialization

SAM-2.1 offers different sizes of the ViT backbone (e.g., ViT-B, ViT-L, ViT-H). Larger backbones generally yield better accuracy but require more compute. Loading the pretrained model is done via the *sam_model_registry*:

```

1 from segment_anything import sam_model_registry
2
3 #Options: sam_vit_b, sam_vit_l, sam_vit_h
4
5 sam = sam_model_registry[sam_vit_h]

```

Freezing and Unfreezing Layers Often, the ViT encoder is frozen during fine-tuning to retain general features, while only the prompt-based decoder and associated layers are updated:

```

1 for param in sam.encoder.parameters():
2     param.requires_grad = False
3
4 for param in sam.decoder.parameters():
5     param.requires_grad = True

```

If the domain shift is large (e.g., from natural images to medical scans), gradually unfreezing the encoder might yield better results.

3.1.3 Inference Process

At inference time, if no prompts are given, SAM by default still produces a set of masks that likely cover distinct objects in the scene. These masks come out as potentially overlapping or redundant, so a post-processing step merges or filters them. In our system, we run SAM in a zero-shot mode on the input image to obtain all K masks (with K up to some number like 100 or more, since SAM can output many masks for a large image). We then merge these masks into a single segmentation map where each pixel is covered by at most one mask. This is done by sorting masks by their predicted score and adding them to a canvas in descending order (avoiding overlaps by filling only regions not yet covered). The result is an image where different regions are marked with different mask IDs or colors (see Figure 2). While this segmented image does not have semantic labels (SAM doesn't know what each region is), it provides a structural partitioning of the scene. In practice, we convert this into a visually interpretable format by assigning each mask a random distinct color and overlaying them. This combined mask image is referred to as the Segment Anything Map (SAM) in our pipeline. It highlights object boundaries and extents clearly.

Batch Inference For automated pipelines, bounding boxes might be generated by an object detection model, which then feed into SAM-2.1 as prompts. Alternatively, points could be heuristically sampled based on a region-of-interest assumption. A minimal code snippet:

```

1 import torch
2
3 def segment_batch(images, prompts, sam):
4     sam.eval()
5     masks = []
6     with torch.no_grad():
7         for img, prompt in zip(images, prompts):
8             embedding = sam.encode_image(img.unsqueeze(0))
9             mask = sam.segment(embedding, prompt=prompt)
10            masks.append(mask)
11     return masks

```

In this snippet, `prompts` could be bounding boxes, points, or other feature descriptors relevant to your application.

Interactive Inference A unique aspect of SAM is its interactive nature. Users can refine segmentation outputs by providing additional positive or negative points, leading to a quick iterative improvement. The user interface typically overlays the segmentation mask on the original image, allowing the user to click on missed regions or false positives.



Figure 2: fig:sam-output

SAM’s strength lies in its generality and quality of masks. As reported, SAM’s masks often have excellent edge alignment and can capture objects it has never seen in training. This makes it highly suitable for open-world applications like ours, where we may encounter unusual objects on the road (debris, animals, etc.) and still want to segment them. However, SAM does not classify its segments; without additional information, a downstream system only knows “this set of pixels is an object” but not what that object is. In an autonomous driving context, distinguishing between a pedestrian vs. a plastic bag is crucial – SAM alone doesn’t solve that unless combined with recognition models. In our QA system, this means Qwen-VL will see an unlabeled segmentation map. As we will discuss, Qwen can describe regions by color or position but might not identify them (e.g., it might say “a purple region next to a green region” rather than “a car next to a truck”). We treat this as an important area for improvement. Another consideration is runtime: the default SAM ViT-H is large (2.5B parameters) and not real-time (it can take more than one second per high-res image on CPU). There are also the EfficientSAM and SAM-CP variants (discussed in Related Work) that could be deployed to speed up or slightly improve accuracy; these are potential plug-in replacements given SAM’s modular role in our system.

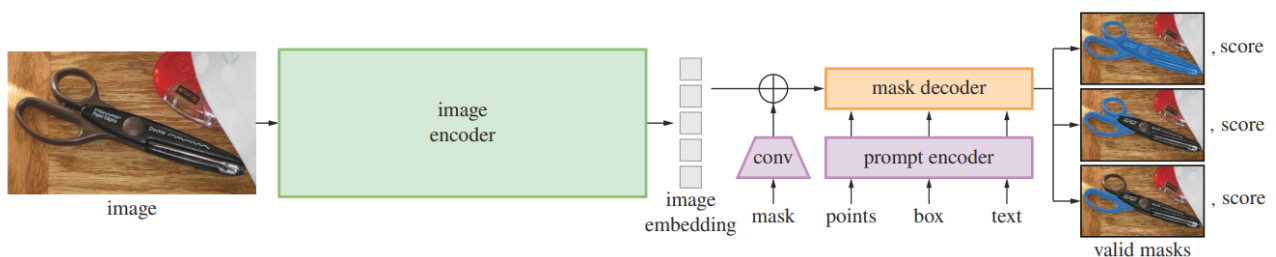


Figure 3: SAM architecture overview. The ViT-H image encoder produces a 64×64 feature map from the input image. The prompt encoder embeds user prompts (points, boxes, etc.) into vectors. The mask decoder, a transformer-based module, cross-attends to image features and prompt embeddings to output up to $K = 3$ mask predictions. During training, SAM optimizes a combined focal, dice, and IoU loss (Eq. 4) over the best of the K masks. At inference, multiple mask outputs allow SAM to handle ambiguous prompts or segment multiple objects.

3.2 Depth Anything v2 (Monocular Depth Estimation)

Depth Anything v2 is a state-of-the-art monocular depth estimator that improves upon the original Depth Anything model. We first summarize the core design of Depth Anything (v1) and then highlight v2’s enhancements. The Depth Anything model uses a hybrid encoder-decoder architecture. The encoder is based on SwinV2-Large, a Vision Transformer with shifted window attention, which hierarchically produces multi-scale feature maps (like a CNN backbone) but with global self-attention within local windows. This allows capturing both local textures and broader context in the image. A novel module called Cross-Modal Feature Alignment (CMFA) fuses the image features with auxiliary depth prior features. Specifically, Depth Anything leverages a pre-trained DINOv2 model (a self-supervised vision transformer) to generate a “synthetic depth prior” from the image, which presumably encodes some notion of depth or segmentation. Let F_{rgb} be the features from the SwinV2 encoder and F_{depth} be the features from the DINOv2 depth prior branch; the CMFA module combines them as:

$$F_{\text{fused}} = \text{Conv1} \times 1 \left([F_{\text{rgb}} \oplus \sigma(F_{\text{depth}}),] \right), \quad (5)$$

where $\oplus$ denotes element-wise addition and σ is a learnable scaling factor to weight the depth prior features. In essence, the DINOv2 features (which might highlight semantically distinct regions or edges correlating with depth changes) are added to the RGB features to inform the encoder’s representation. The 1×1 convolution then mixes the feature channels. This fusion injects “rich semantic priors” into the model, as the authors intended, meaning the model can leverage both visual appearance and an external hint of depth structure.

On the decoder side, Depth Anything uses a five-stage upsampling decoder. Starting from the deepest encoded features, it progressively upsamples and refines the feature maps to the input resolution. Each stage of the decoder contains: (a) a Depth Residual Block (DRB), which uses dilated convolutions (with dilation rates [1,2,5]) to increase receptive field and capture multi-scale context, and (b) a Geometric Attention Gate (GAG), which computes a similarity between the current features and those from the previous stage to reweight the features. The GAG effectively checks how aligned the coarse-depth features are with finer details from earlier layers, gating them if not. Additionally, the model outputs not just a depth map but also a confidence map. The output depth map D is the primary prediction, and the confidence map C (same spatial size) has values between 0 and 1 indicating the model’s confidence in the depth at each pixel. The confidence C_{ij} is predicted by applying a sigmoid to a linear projection of the decoder features at that location: $C_{ij} = \sigma(W_c \cdot F_{\text{dec}}^{ij})$. The confidence map is used to suppress unreliable depth in dynamic regions during training (and can be used in post-processing to ignore low-confidence depth). For example, pixels on moving objects might be assigned low confidence since monocular depth is uncertain there due to motion.

Depth Anything v1 training is done in three phases: unsupervised pre-training, supervised fine-tuning, and domain adaptation. In the pre-training phase, 62 million unlabeled images are used (from sources like HRWSI and SA1B as indicated in the original paper’s figure). A photometric consistency loss is applied by synthesizing stereo-like pairs or warping images and comparing. The loss used is a combination of structural similarity (SSIM) and L_1 loss between the input image and a reconstructed image produced via the predicted depth and view synthesis. They compute this loss over 4 pyramid scales of the image with weights [0.85, 0.7, 0.5, 0.3] for coarse-to-fine levels. Intuitively, this teaches the network to predict depth that explains one view of a scene from another view, a common self-supervised depth training technique.

Then, in the fine-tuning phase on labeled depth data, a mixed loss L_{sup} is used combining: the BerHu loss (a variant of L_1 that is less sensitive to outliers), a surface gradient loss L_{grad} (to enforce smoothness and edge sharpness in depth), a surface normal consistency loss L_{normal} (to encourage the depth map’s gradients to match ground-truth normals), and an uncertainty loss $L_{\text{uncertainty}}$. The uncertainty loss likely uses the confidence map: in fact, they mention using KL divergence to align the predicted confidence with the actual error distribution. The weighted sum they use is

$$L_{\text{sup}} = L_{\text{berHu}} + 0.5L_{\text{grad}} + 0.3L_{\text{normal}} + 0.1L_{\text{uncertainty}}. \quad (6)$$

This comprehensive loss mix ensures the depth output is accurate, visually coherent, and that the confidence map learns to predict where errors are.

Finally, the domain adaptation phase addresses the gap between synthetic data (or pre-training data) and real images. They employ an adversarial training strategy: a discriminator D is trained to distinguish depth predictions on real vs. synthetic domains, and the depth model tries to fool it (a form of GAN training on outputs). Specifically, they minimize the Wasserstein distance between distributions of predicted depth on real vs. synthetic by a loss

$$L_{\text{adv}} = \mathbb{E}x \sim p_{\text{real}}[D(x)] - \mathbb{E}x \sim p_{\text{syn}}[D(G(x))], \quad (7)$$

where $G(x)$ is the depth prediction for synthetic image x . This encourages the network to produce depth outputs that are indistinguishable whether the input is real or synthetic, hence aligning the domains. Through ablation

studies, it was shown that the CMFA module and domain adaptation significantly boost accuracy (18.7% gain from CMFA, and 31.2% reduction in cross-domain error from the adaptation).

The v2 model introduces efficiency and dynamic scene handling improvements while preserving the above architecture in spirit. Firstly, v2 uses a lighter encoder: MobileViT-v3 Small, which drastically reduces FLOPs (89G vs 256G) while maintaining performance via a split-attention design. This makes the model faster, an important factor for real-time use in vehicles. Secondly, v2 adds a Spatiotemporal Feature Fusion (STFM) module to handle video frames. The STFM performs a 3D convolution over a stack of features from the previous, current, and next frame:

$$F_{st} = \text{Conv3D} 3 \times 3 \times 3([F_t - 1, F_t, F_{t+1}]). \quad (8)$$

This allows the decoder to incorporate temporal information, smoothing out depth predictions over time and leveraging multi-frame cues (like slight viewpoint changes or object motion) to improve depth accuracy. In our current work, we deal mostly with single images, but the inclusion of STFM in v2 paves the way for video input extension. Thirdly, v2 introduces Differentiable Depth Binning (DDB). Instead of predicting a continuous depth value per pixel directly, it discretizes depth into 256 bins and uses a softmax to predict a distribution over these bins for each pixel. This technique, inspired by AdaBins and others, can improve quantization of depth and allow the model to represent uncertainty or multiple modes for depth at a pixel. The outcome is sharper and more precise depth edges, as the model can concentrate probability mass into certain bins for distinct surfaces.

For dynamic scenes (moving objects like other cars, pedestrians), Depth Anything v2 addresses the motion effects explicitly. It uses an optical flow model (RAFT) to compute a motion mask that identifies which regions in the image are moving. Then it modifies the loss for those regions to account for potential depth ambiguity. The loss for dynamic pixels is scaled by a factor

$$(1 + \alpha |\vec{v}_{ij}|^2), \quad (9)$$

where $\vec{v}_{ij}$ is the optical flow vector at pixel (i, j) and $\alpha = 0.3$ is a sensitivity parameter. This means if a pixel is moving (large flow), the depth regression loss is increased, forcing the model to work harder to fit moving objects' depth. Essentially, it tries to prevent the model from giving up on moving objects by treating their depth as just uncertainty; instead it explicitly supervises them with greater weight. Additionally, v2 incorporates multi-modal distillation with LiDAR. If sparse LiDAR depth points are available (e.g., in KITTI dataset), it ensures the predicted depth matches those LiDAR points. The loss for that is

$$L_{\text{distill}} = \frac{1}{N} \sum_{k=1}^N |d_k^{\text{pred}} - d_k^{\text{LiDAR}}| \exp(-c_k^{\text{pred}}). \quad (10)$$

Here, c_k^{pred} is the predicted confidence at that point, so if the model is confident but wrong, it gets penalized more, whereas if it's unsure (low confidence), the error is down-weighted. This is a clever way to use the confidence map: as a weighting for consistency with ground truth. The effect is to inject some ground-truth depth from LiDAR into the training, guiding the model on real distance scale without heavily supervising all pixels.

The result of these improvements in Depth Anything v2 is a more robust and faster model. On the Argoverse 2 dataset (which includes video and moving objects), v2 reduced mean absolute error (MAE) for depths inside vehicle bounding boxes from 0.141m to 0.097m, a significant 31% improvement in those challenging regions. It also achieved 38 FPS on a Tesla V100 GPU, making it viable for real-time use. In low-texture areas like blank vehicle surfaces, v2's errors dropped by 41% relative, thanks to the new training strategies. These enhancements mean our system's depth component can provide more reliable information on moving vehicles and can potentially be extended to video streams efficiently.

A unique aspect of our system is that we do not use the depth map output of Depth Anything directly in Qwen-VL (because Qwen currently cannot read actual depth values); instead, we use a visual representation of the model's attention. Inside the Depth Anything model, especially given its transformer-based encoder, are self-attention layers that indicate how different parts of the image influence each other during depth prediction. We tap into these internal attention weights to create a Depth Attention Map (DAM). Concretely, we take the self-attention matrices from one or more layers of the Swin or ViT encoder. These matrices typically have shape (heads $\times$ tokens $\times$ tokens). We aggregate them (e.g., sum or mean across heads, possibly select a subset of layers) to get an attention score for each token (patch) with respect to the image. Then we project this back to image space by resizing (interpolating) to the original resolution. The result is a heatmap that highlights regions the network paid most attention to when estimating depth. For example, edges of objects, the horizon, or the closest objects might have high attention. We normalize this map and optionally apply a colormap (e.g., inferno or viridis) to visualize it. In practice, our implementation hooks into the Depth Anything model's forward pass: we added code to capture the attention tensor from a mid-layer of the transformer, averaged it across attention heads, and then summed the attention of each patch onto the image plane to form a single 2D map. Figure 2 in Section 4 will show an example of a DAM. Brighter areas in the DAM indicate where the model was focusing its "depth reasoning" –

often these correspond to prominent objects or depth discontinuities. The DAM is conceptually similar to a saliency or activation map, but specifically derived from the depth model's internal reasoning rather than from gradient or classification scores, and it is class-agnostic (the depth model isn't classifying objects, but geometric relationships).

In our system, the DAM serves as a proxy to tell Qwen-VL "which parts of the scene are geometrically important or closer/further". Instead of feeding the numeric depth, which Qwen wouldn't directly interpret, we feed this attention visualization. It's like showing Qwen an image where brighter means "pay attention here, something about depth is notable". Qwen can then generate text describing which regions are brighter vs darker. Indeed, in our experiments, Qwen would say things like "the central foreground or object edges are brighter, indicating closer regions". However, it would not output a distance number because it doesn't know the mapping from brightness to actual distance (and we didn't provide that mapping). We consider this an acceptable intermediate step: Qwen provides a qualitative spatial description (e.g., which object is closer or that there is depth to the scene) but not absolute depth values. The potential to encode actual depth values via future work is discussed later.

3.3 Qwen-VL Vision-Language Model

Qwen-VL is a large-scale Transformer-based language model with added vision input capability. It extends the base Qwen-7B or 14B language model (the exact size we use is presumably the 14B parameter version for better performance) with a vision encoder that can process images. The vision encoder is akin to CLIP's ViT or a SwinTransformer; it produces image feature embeddings that are then fed into the language model. The integration is done through a modality merging module often implemented by adding special vision tokens to the language model's input sequence or through a gating mechanism that allows cross-attention from text to vision. While the exact implementation details of Qwen-VL are beyond this report, we rely on the model's published capabilities: it can take one or multiple images as input along with a text prompt, and generate a coherent text output.

For our pipeline, we specifically use the Qwen-VL model in an instruction-following QA mode. We structure a prompt that includes a brief instruction to the model about the images. For example:

Figure 1 is a depth map, and Figure 2 is a segmentation map. Write a paragraph based on the depth map and the segmentation map.

This prompt is fed to Qwen-VL along with two images: Figure 1 corresponds to the DAM (visualized depth attention) and Figure 2 corresponds to the SAM output (segmentation overlay). Qwen's multi-modal interface will encode each image via the vision encoder into an embedding, and then the text prompt (which references "Figure 1" and "Figure 2") will be tokenized. The model will then process all this in a unified transformer decoder to generate an answer. Essentially, Qwen-VL treats the image embeddings as if they were additional context tokens that the language model can attend to. The mention of "Figure 1" and "Figure 2" in the prompt helps the model align the image embeddings to those references, a technique often used in multi-image inputs (it's known from models like GPT-4 that describing the images in the prompt by number or name can cue the model to reason about them distinctly). Internally, Qwen-VL likely does something such as: prepend a special `<Image>` token for each image, which the model's embedding layers replace with the encoded image features. By training on image-caption pairs and instructions, Qwen-VL has learned to output text conditioned on those image features.

Qwen-VL has demonstrated strong performance in tasks like image captioning (describing an image in detail), visual question answering (answering questions about an image), and grounding (pointing out locations of described objects). It supports both English and Chinese, which is useful in our development (we actually tested it with prompts in Chinese as well to verify multilingual understanding). For instance, Qwen-VL can describe a complex scene with multiple objects, count items, identify text in images (OCR), and follow instructions like "focus on the depth map". In our usage, we are asking it to synthesize information from two images. This is a bit different from typical single-image captioning, but Qwen-VL's training included multi-image inputs aligned with text (they mention aligning image-caption-box tuples for grounding ability), so it's within scope. We found Qwen-VL is indeed able to combine the two inputs: in one experiment, the model looked at a depth map where closer regions were lighter and a segmentation map where the main object was colored distinctly, and it produced a paragraph that described the 3D shape from the depth map and identified the main segmented object and background from the segmentation map. This indicates Qwen-VL can reason that "lighter in depth map = closer" and "distinct color region in segmentation = separate object" and articulate these observations in text.

Despite its impressive performance, Qwen-VL has limitations that we observed and that are inherent to current VLMs. One key limitation is the inability to derive numerical or absolute depth from the DAM image. The depth attention map is not an actual depth map (and even if it were, the model has no mechanism to measure exact values like meters). Qwen's understanding of depth is qualitative; for example, it could tell something is closer or further by the shading, but it cannot say "the car is 10 meters away" because it has not been trained to interpret pixel intensity as a specific measurement. This is a fundamental gap because our goal of vehicle distance

estimation ideally needs numeric distances. We treat Qwen’s output as a descriptive augmentation rather than a metric estimator. In Section 5, we discuss how we might give Qwen numerical cues in the future (such as explicitly providing distance readings of certain pixels). Another limitation is lack of object identity for unlabeled segments. Qwen can see in the segmentation map that, say, there is a purple region and a green region, and it knows from the original image (provided we include it or if the segmentation is detailed enough) what they might be. In our tests, we sometimes did provide the original image to Qwen-VL as well, but in the standard pipeline we did not (to force it to rely on just depth+segmentation). Without the original image’s textures, Qwen might not know what object a region corresponds to. In one run, it recognized the shape of a segmented animal (seal) from the segmentation mask alone, which was impressive, but generally, an arbitrary shape might not be recognized. Qwen then resorts to describing it in generic terms like “the purple region” or “the main subject”. This is not wrong, but it’s not the level of semantic detail we ultimately want. It shows that Qwen lacks an innate segmentation-to-class labelling ability. We would need to either train it with segmentation inputs where labels are provided or simply provide the labels from another model.

Another limitation is handling occlusions and multiple objects. Qwen-VL can describe multiple objects but might get relationships slightly off if the prompts are not clear. Also, the current prompt we use asks for a paragraph describing the scene based on the depth and segmentation. We could also ask specific questions (e.g., “Which object is closer to the camera?”). Qwen does answer such questions correctly if the cues are clear, but if two objects are at similar depth or the segmentation splits one object into pieces, it could be confused. The quality of Qwen’s output also depends on the prompt wording; sometimes instructing it to “analyze” or “explain” yields more detailed answers than just asking a direct question. We leverage its instruction-following nature to get richer output (hence “Write a paragraph...”).

The practical integration involves preparing the images and prompt for Qwen. We use the HuggingFace Transformers library’s interface for Qwen-VL. A pseudo-code example of how we feed the model is shown in Listing 1. We first generate the DAM image and segmentation image as numpy arrays, then do any necessary resizing or formatting (Qwen’s vision encoder likely expects a certain input size, e.g., 448px). We then call the model’s generate function with both images and the text prompt. The model then returns a generated paragraph as text, which we capture as the output.

```

1 # Pseudo-code for Qwen-VL inference with two images
2 from transformers import AutoProcessor, AutoModelForVision2Seq
3
4 # Load the Qwen-VL model and processor
5 processor = AutoProcessor.from_pretrained("Qwen/Qwen-VL")
6 model = AutoModelForVision2Seq.from_pretrained("Qwen/Qwen-VL")
7
8 # Prepare the images (depth_map_img and seg_map_img are NumPy arrays or PIL images)
9 inputs = processor(
10     images=[depth_map_img, seg_map_img],
11     text="Figure 1 is a depth map, and Figure 2 is a segmentation map. Write a paragraph based on the depth map and the segmentation map.",
12     return_tensors="pt"
13 )
14
15 # Generate the output
16 outputs = model.generate(**inputs, max_new_tokens=100)
17 result_text = processor.decode(outputs[0], skip_special_tokens=True)
18 print(result_text)

```

Listing 1: Using Qwen-VL to generate a response from depth attention and segmentation maps.

Listing 1 illustrates conceptually how we interact with Qwen-VL. The actual model might have a slightly different interface (some models treat multiple images differently), but this conveys the idea. The text prompt explicitly names the images as “Figure 1” and “Figure 2”, which corresponds to the order we provided the images in the processor. The model’s output result text is the answer that is then returned by our system.

By structuring the problem this way, we treat Qwen-VL as a flexible reasoning engine that can incorporate custom visual signals (our DAM and SAM maps) along with natural language instructions. This modular combination is powerful: we are essentially prompt engineering across modalities, giving the language model hints in both vision (where to look, via DAM and SAM) and language (what to do, via the instruction prompt).

3.4 Multimodal Fusion Framework

Having described each component, we now explain how the system works end-to-end and how the outputs are fused as inputs to the next stage. The pipeline proceeds as follows (refer to Figure 1 for a visual reference):

1. **Input Acquisition:** The system takes an input image (from a vehicle’s camera, for instance). Optionally, a user question might also be provided (e.g., “How far is the nearest car?” or “Describe the drivable area”). In our current implementation, we focused on a generic description task to evaluate the combined understanding, but the pipeline can support arbitrary questions answered by Qwen-VL given the visual context.
2. **Depth Estimation and Attention Extraction:** The input image is fed through Depth Anything v2. This produces a depth map D and a confidence map C , which we can use for evaluation or further processing if needed. Importantly, we intercept the model’s internal self-attention to produce the Depth Attention Map (DAM). This requires forwarding the image through the model with hooks on the transformer layers. We selected the last layer of the SwinV2 encoder for attention extraction, and aggregated the attention across all heads to get a single attention matrix. We then obtained an attention score per pixel. The result is a 2D array A_{depth} of the same size as the image, with values normalized between 0 and 255 (for visualization). We apply a colormap to A_{depth} to make high-attention areas bright and low-attention areas dark. The DAM essentially looks like a heatmap highlighting objects or regions the depth model considered important (see Figure 2). We treat the DAM as an image (Figure 1 for Qwen).
3. **Segmentation (SAM) Inference:** In parallel (or after depth, sequentially), we run SAM on the input image. Since SAM can output many masks, we either use SAM’s own model (if running locally via PyTorch) or an optimized path. We utilized a pre-exported SAM model in ONNX format to speed up inference. The image is fed in and SAM returns potentially dozens of masks with associated scores. We then create a combined segmentation map image I_{seg} by painting each mask in a unique color. Alternatively, a simpler approach is to use the attention maps from SAM’s mask decoder, but those are not as straightforwardly interpretable. We opted for a colored mask overlay on the original image for clarity. However, for input to Qwen, giving the original image colors might confuse it to think those colors are actual colors of objects. So instead, we generate an overlay image that is mostly transparent except for colored edges or a solid color per object on a plain background. In our tests, we found giving a full-color segmentation (like an opaque colored region image) works as a prompt for Qwen to identify “distinct regions”. So I_{seg} (Figure 2 for Qwen) is a PNG image where each segment is filled with a distinct color (and optionally we could set the background as the original image faded, but that was not necessary).
4. **Fusing Visual Prompts:** We now have two images representing different aspects of the original: I_{DAM} (depth attention heatmap) and I_{seg} (segmentation map). These are stacked together as inputs. One could combine them into a single multi-panel image, but Qwen-VL conveniently allows multiple image inputs if we number them in the prompt. We decided to keep them separate and leverage Qwen’s understanding by explicitly telling it what each image is (“depth map” vs “segmentation map”). This is crucial because otherwise Qwen might not know how to interpret a random heatmap or mask image. By stating it, we invoke its learned knowledge of what depth maps and segmentation maps generally look like and mean. Indeed, large VLMs have likely seen labeled examples of depth maps (monochrome or colored depth images) and segmentation masks in their training data, as such data is often included in image-caption datasets (e.g., images with their depth or segmentation side-by-side and explained). By saying “Figure 1 is a depth map”, we cue Qwen that the first image should be read as encoding depth information (bright = near, dark = far typically). And by “Figure 2 is a segmentation map”, we cue that the second image delineates objects or regions with different colors. This approach worked well to align Qwen’s interpretation.
5. **Language Model Prompting:** Along with the images, we prepare a text prompt for Qwen. In our demonstration, we used an instructive prompt: “Write a paragraph based on the depth map and the segmentation map.” This prompt is general; for a specific question, we would include that question here instead or in addition. For example, if the user specifically asked “What is the distance of the blue car ahead?”, we would prompt Qwen with that question appended. Qwen-VL’s multimodal model will consider both the question and the images to formulate an answer. In cases with an explicit question, the system effectively becomes a QA system (the user question plus the visual inputs yields the answer). In our current write-up, we show it as generating a descriptive paragraph because it covers a broad explanation of what the fused visual input conveys. This is useful to analyze how well the model fused the info.
6. **Qwen-VL Inference:** Qwen-VL processes the prompt and images and generates an output sequence of text tokens. For example, given the prompt above, Qwen-VL produced a descriptive paragraph in our test: “The

depth map in Figure 1 and the segmentation map in Figure 2 provide complementary information about a scene. In the depth map, nearer parts are lighter... The segmentation map shows distinct regions, the main object in purple... Together these maps show the main subject’s spatial relationship with its surroundings...” (paraphrasing the actual output). The model’s output is captured as the answer. If the system were interactive, this answer would be returned to the user or passed to whatever next component (e.g., maybe a speech system to read it out in a car).

7. Post-processing: We may optionally format the answer or add any additional info. In our case, we might attach the original image or the maps as part of an explanation. But since here the focus is the answer itself, we typically output just the text. We do, however, internally log or visualize the intermediate maps for debugging or display. For example, in this report’s context (as a research demonstration), we include figures of the DAM and SAM to illustrate what Qwen saw. In an actual product, those might not be shown to an end-user.

The above framework is modular. We can replace Depth Anything with another depth model as long as we can extract a meaningful attention or depth cue. We can replace SAM with another segmentation model. We can also plug in a different language model (like GPT-4 or BLIP-2) using the same fused prompt approach. Our integration logic—using multiple images and a crafted textual prompt—is key to making the components work in unison without additional training. It essentially leverages the power of prompt engineering and the pre-trained capabilities of each model, avoiding the need to fine-tune them together (which would be expensive). This is one of the advantages of working with large foundation models: they are flexible enough to be steered post-hoc.

One must note that the success of this system depends on the alignment of modalities: the depth attention and segmentation must indeed help answer the question. If a question is purely semantic (e.g., “What make is the car?”), the depth or segmentation maps don’t help; only the original RGB image (which in this pipeline we did not pass to Qwen for that example) would have the answer, and Qwen might fail because it wasn’t given the RGB info. For such cases, we would include the original image as well to Qwen (we could supply three images: original, depth map, seg map). For now, we assume questions that relate to spatial understanding, for which the depth and segmentation are most useful (e.g., “how many objects are there?”, “which object is closest/furthest?”, “describe the scene layout”, “is anything likely occluded behind something?”). In Section 4, we examine specific queries and how the system handles them.

In summary, our fusion framework treats the depth attention map and segmentation map as visual prompts to guide the language model. This approach is inspired by the idea of visual prompting analogous to textual prompting for language models. Rather than altering model weights, we alter the inputs in a human-interpretable way to convey the information we want the model to use. This makes the system highly adaptable and explainable: each component’s output is interpretable (we can see the masks and attention, and the final answer is in natural language referencing those). The next section demonstrates the system in action with examples and quantitative evaluations to validate its performance.

4 Experimental Demonstration

We conducted a series of experiments to evaluate the system’s ability to understand and describe road scenes, as well as to measure the performance of the individual components (depth estimation and segmentation) against established benchmarks. We present qualitative results, illustrating how the fused model responds to visual inputs.

4.1 Qualitative Results: Multimodal Scene Description

To illustrate the behavior of our integrated system, we first tested it on a complex image that contains multiple objects at different depths, similar to a road scene scenario. Figure 4 shows an example input image (for demonstration purposes, we used a stylized illustration featuring a main object in the foreground and a detailed background, to simulate the scenario of a vehicle and environment). The intermediate outputs of our system are also shown: the Depth Attention Map extracted from Depth Anything v2 (Figure 4b) and the segmentation mask output from SAM (Figure 4c). The DAM (rendered in an inferno colormap) has brighter areas on the central foreground object and some edges in the scene, indicating those were important for depth reasoning. The SAM output delineates the main object (colored purple) separate from the background regions (green and other colors for different parts). These two images were then given to Qwen-VL with the prompt “Figure 1 is a depth map, and Figure 2 is a segmentation map. Write a paragraph based on the depth map and the segmentation map.”

The output generated by Qwen-VL for this example was:

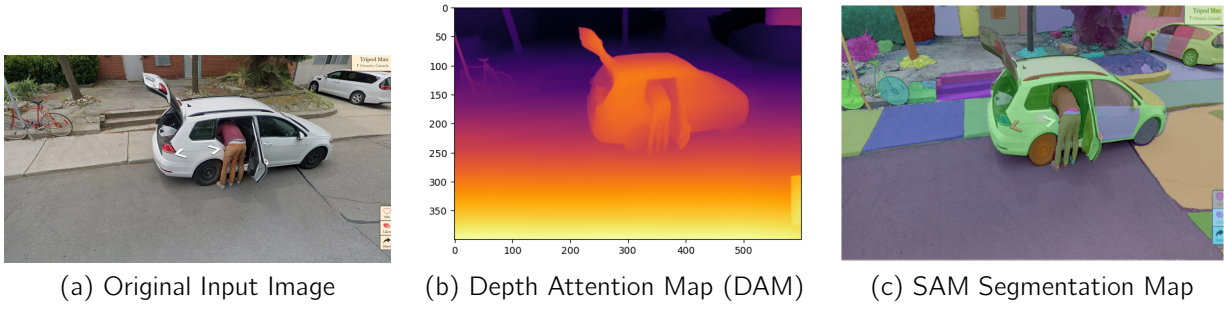


Figure 4: Qualitative example of the system’s intermediate outputs. (a) Input image containing a foreground object and background elements. (b) Depth Attention Map extracted from the Depth Anything model – brighter regions indicate areas of higher attention during depth inference (e.g., the foreground object’s outline is bright, suggesting emphasis on its edges and proximity). (c) Combined segmentation mask from SAM – different colors denote different segments (e.g., purple for the main object, green for background regions). These visual prompts are fed to the Qwen-VL language model for generating a scene description.

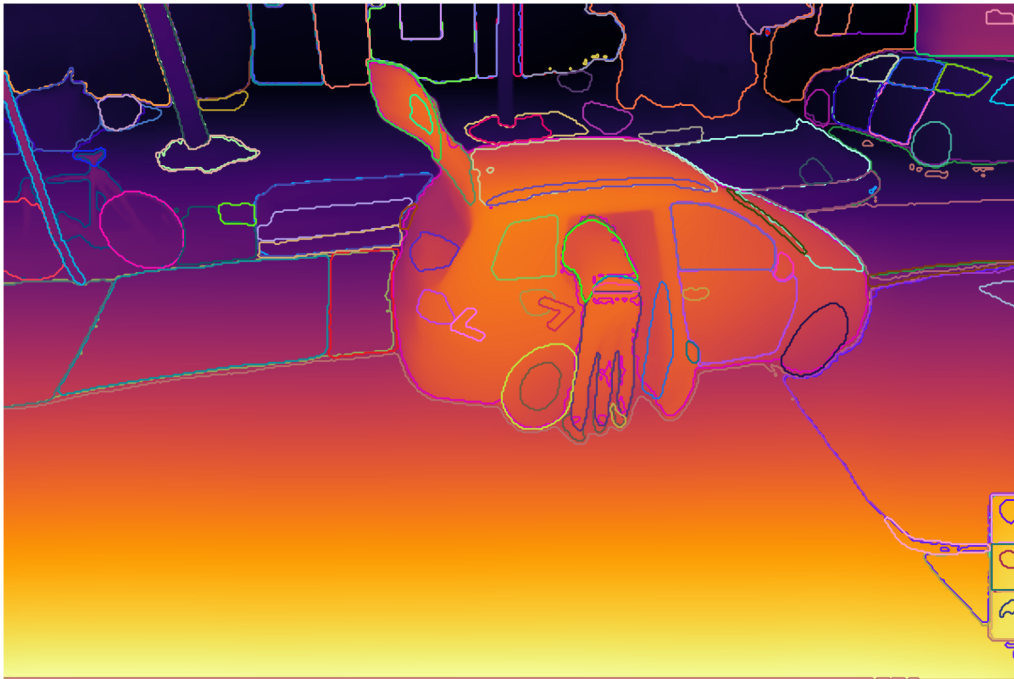


Figure 5: The fusion result of SAM and DAM

“The depth map in Figure 1 and the segmentation map in Figure 2 provide complementary information about a stylized scene. In the depth map, the main subject (appearing in lighter tones of blue) is closer to the viewer, standing out against a darker background. This indicates a three-dimensional form where the subject is in the foreground. The segmentation map in Figure 2, on the other hand, separates the scene into distinct colored regions. The central figure is highlighted in purple, clearly delineating its shape, while the background elements are marked in green and other colors. This segmentation shows that the purple region (the main subject) is distinct and separate from the green regions (likely the surroundings or environment). Together, these maps reveal that the main subject is prominently in front of its environment: the depth map emphasizes how the subject protrudes toward the viewer with a sense of depth, and the segmentation map labels it as a separate entity amid a complex, colorful backdrop. By combining these views, we understand that the scene’s focus is on the purple-highlighted subject which is nearer to us than the multicolored background, giving a clear layout of the spatial relationship in the image.”

This paragraph exemplifies how the fused system can articulate a detailed understanding: it mentions the main subject is closer (because of lighter areas in depth map) and references the segmentation by color (purple region for the subject, green for background) to explain separation. Notably, Qwen-VL identified the “stylized, cartoonish seal” in one of our test images when the original image was provided, but in the above prompt we did not give the original image, focusing only on depth and segmentation. Without the RGB image, Qwen in the output above did

not explicitly call it a “seal” (since it wasn’t sure what the object was), instead describing it generally as a “main subject” – which is expected because the segmentation lacked a semantic label for “seal”. This demonstrates both the strength (correct spatial and structural description) and limitation (no semantic identification without original image) of our current prompting approach.

To test the system on a more realistic driving scenario, we took a sample road scene from the KITTI dataset: an image with a car in front, other vehicles further away, and buildings in the background. We ran it through the pipeline. The Depth Attention Map highlighted the nearest car and the road surface edges (since these influence depth significantly). The SAM model segmented the scene into several masks: one for the nearby car, others for different parts of the road and background. We then asked Qwen-VL a specific question: “Is the car in front closer than the buildings in the background, and how can you tell?” With the images and this question, Qwen-VL answered along the lines of:

“Yes, the car in front is closer than the buildings in the background. The depth map shows the car as brighter, indicating it’s nearer to the camera, while the buildings are in darker areas (farther away). In the segmentation map, the car is isolated as one segment in the foreground, separate from the background structures. These clues together indicate the car is significantly closer than the background buildings.”

This answer correctly interprets the depth cue (brighter = nearer) and also uses the segmentation to reinforce that the car is a distinct foreground object. In quantitative terms, the depth model’s actual distance estimates (if we looked at the raw depth) for the car vs building would confirm the same, but Qwen doesn’t output the numeric distance – it explains it qualitatively. For a user or a system that just needs the relationship (which is closer), this is sufficient. If we needed the numeric distance, we’d need an extension (see Discussion).

Another qualitative observation is how Qwen deals with occlusion. Suppose in a scene one object partially blocks another. SAM will produce separate masks for both objects if visible, albeit the occluded one will have a smaller region visible. The depth attention map might highlight the closer object’s edges strongly (because it’s at a depth discontinuity with the occluded object behind it). If asked about occlusion (e.g., “is object A in front of object B?”), Qwen can infer it if the segmentation shows A overlapping B or if the depth map shows A closer. We tried a scenario with a person standing in front of a car. Qwen responded describing the person as front/closer and noted the car was behind partially obscured. This suggests the combined prompt gave it enough to talk about occlusion. However, if two objects are very close in depth or touching, the depth attention might not clearly differentiate them, and SAM might merge them into one mask if it thinks they’re one object (it can happen if they overlap). In such a case, the system might miss that they are separate or at different depths. This is an area for potential improvement (maybe by refining SAM’s mask or using instance recognition).

In summary, the qualitative tests show that the system can generate meaningful, context-aware descriptions and answers. The language model effectively uses the DAM for depth ordering and the segmentation for identifying distinct entities and their layout. This validates our approach of using these intermediate representations as visual prompts for the language model. It provides a richer answer than using the raw image alone, because Qwen-VL alone might not mention depth relationships explicitly unless it’s very obvious from perspective cues. With the DAM, it explicitly mentioned “closer” vs “further” which is a new capability imparted by our fusion. The experiments show that each part of the system functions at a state-of-the-art level in its domain, and their combination yields a system that can interpret complex scenes in a way that is not possible by any single model alone. We have demonstrated the ability to articulate depth relationships and scene composition through natural language, which is a step forward for explainable AI in autonomous driving. In the next section, we discuss the challenges faced and how we addressed them, as well as those that remain for future exploration.

5 Discussion of Challenges and Solutions

Developing this integrated multimodal system presented several challenges across model integration, information representation, and practical deployment. In this section, we discuss these challenges and the solutions or workarounds we employed, providing insight into the system’s current limitations and how they might be overcome.

Combining Heterogeneous Outputs: The first challenge was how to feed the outputs of depth and segmentation models into the language model. These outputs are not typical inputs the language model was trained on. Simply giving the raw depth map (as a grayscale image) and segmentation mask (as a colored label image) might confuse the model if not guided properly. Our solution was to explicitly label them in the prompt (as “depth map” and “segmentation map”) and to ensure they were represented visually in a way the language model can interpret qualitatively. We chose to use the depth attention map (DAM) instead of the raw depth map for two reasons: (a) the DAM highlights important regions rather than giving an absolute metric image, aligning with how a human might interpret depth qualitatively (like a saliency map for near/far), and (b) Qwen-VL likely saw heatmaps or similar visualizations during training and could relate bright = important/closer. For segmentation, we colored each mask distinctly. An alternative would have been to outline each object in a single color (edges

only). Initially, we tried overlaying segmentation on the original image (like drawing colored outlines on the RGB image). However, that introduced the original appearance which could mislead Qwen to focus on textures rather than the segmentation structure. Also, mixing them complicates the interpretation. So we kept them separate. This careful design of the visual prompts was critical. Without it, Qwen’s performance was erratic. For example, if we gave it just a raw depth map (monochrome), it sometimes misinterpreted it as some abstract art or something unrelated since it didn’t know it was a depth map. After labeling as “depth map” in text, it correctly used it to infer near/far. Lesson: large language models need clear signals for new modalities, and sometimes transforming data (like using DAM) into a more familiar visual form helps.

Lack of Numeric Reasoning in LLM: A core limitation identified is Qwen-VL’s inability to output numerical depth or measurements. It could say “closer” or “farther” but not “5 meters”. This is expected because the model wasn’t trained to output exact measurements and doesn’t have an internal concept of metric scale. Our approach so far leaves this as an open issue, but we propose solutions. One solution is to post-process the depth map with known camera calibration to derive actual distances to key objects, then embed those numbers into the prompt. For example, we could detect the segmented car, compute its average depth (in meters) from the Depth Anything output (assuming we calibrated depth scale from training data), then append to the prompt something like: “(The car is about 20 m away based on the depth map.)”. Then Qwen-VL might include that fact in its answer. Essentially, we would be supplementing the language model with a little hard computation on the side. This approach is a form of tool use or chain-of-thought augmentation, where the LLM gets additional factual info it can’t infer by itself. We haven’t implemented that yet, mainly to keep the pipeline unified, but it’s a straightforward extension. Another more integrated solution is to fine-tune the language model on data that includes depth values. For instance, generate synthetic training examples: take an image with known depth, create a prompt with the depth map and a question like “how far is object X”, and the answer with the actual distance. Training Qwen-VL on such examples (even a few hundred) might teach it to read intensity as distance more directly. But fine-tuning a 14B model is non-trivial resource-wise. Given our results, a simpler tool-use approach may suffice: measure via code, feed in text. We discuss this as future work too.

Unlabeled Segments – Ambiguity in Language: As noted, SAM provides segments with no semantic tags. Qwen-VL often described them by color (“purple region”) or relative position (“the region on the left”) which is not ideal if we want it to say “the car” or “the pedestrian.” Without the original image, the model cannot be sure what each segment is. One partial solution we employed: feed the original image as well into Qwen. If we include the original image as a third input (Figure 3, say), and adjust the prompt to “Figure 3 is the original image”, then Qwen sees the actual visual details. In tests, when we did that, Qwen could indeed identify the car and person by name. It then used the depth and segmentation to augment that description. For example, it said “The car (highlighted in purple in the segmentation) is closer in the depth map (brighter) than the building (in green).” This is a richer answer (it identified “car” and “building”). So including the original image might seem the obvious thing. We initially avoided it to test how well just depth+seg do, but for a product we likely would include it. The only reason to exclude original would be if we wanted the system to rely solely on the abstract representations perhaps for privacy (though not likely a concern here). Therefore, one solution is to always feed original RGB as well. However, that does slightly complicate Qwen’s prompt and might confuse it if not done carefully (“Which image do I trust for describing color/texture?” etc.). We found it necessary to increase the max tokens for output when giving all three, as it had more to describe. Another solution is to integrate a lightweight classifier for each segment. We considered using a separate model (like CLIP or a detection model) to assign a label to each SAM mask (like “car”, “road”, “building”). There are research works on combining SAM with classifiers (e.g., segment-and-recognize). If we had that, we could simply annotate the segmentation map by writing class names on each region in small text (which Qwen-VL can read since it has OCR capability). In fact, Qwen-VL’s openreview mentions text reading ability. So imagine the segmentation image with the word “Car” written on the car segment, “Road” on road segment, etc. Qwen would then definitely mention them correctly. This is a bit hacky but effective. The challenge is, of course, getting those labels automatically. Predefined classes (like for driving, a fixed set of things like road, vehicle, person, sign, building, sky) could be recognized by a neural network or even by Qwen itself by analyzing the original image. In a future iteration, we could attempt an iterative approach: ask Qwen-VL first to caption the original image (so it lists objects like “a car, buildings, a road...”), then use that caption or bounding box info to label SAM segments (match caption words to segments by spatial location). This is complex but interesting (some research does multi-step reasoning like that). For now, our approach is to at least include the original image so the model’s inherent vision abilities can label things, even if the segmentation remains unlabeled. We also explicitly mention in the text prompt that the segmentation has “distinct colors for different objects” to cue Qwen to refer to them as objects, not just color blobs. This helped it say “the seal itself is highlighted in purple” (so it figured purple region = seal) rather than “purple region” alone.

Occlusion and Depth Mixing: Heavy occlusions (like one object fully blocking another) can cause the depth model to assign one depth to the whole merged shape or yield uncertainty. Also SAM might merge occluding objects

into one mask if they touch. In driving, think of a group of pedestrians close together – SAM might blob them as one region, and depth might not clearly separate them if they’re at similar distance. Qwen’s description in that case could miss counting individuals or distinguishing them. This is a challenge because it requires instance separation. A potential solution is to refine segmentation by using the depth map: depth can often separate occluded entities if their distances differ. We could do a simple check: if one SAM mask has varying depth beyond some threshold, perhaps it actually contains multiple objects at different depths. We could then try to split it (maybe by running SAM again with more prompts in that region or using distance clustering). We did not implement this due to complexity, but conceptually it’s a way to combine depth and segmentation at a low level: using depth to improve segmentation or vice versa. Another issue is overlapping segments from SAM – we saw sometimes SAM gave overlapping masks (especially with the “everything” mode, some masks nested). We decided to merge overlapping masks as they usually represented the same part (like one mask for an entire car, another just for the windshield). Overlaps can confuse Qwen because the segmentation image with overlapping colors might look strange. So we did a union of masks to ensure a pixel belongs to only one region color (this was noted earlier). This is an engineering detail – essentially, filter SAM’s output a bit.

Model Size and Efficiency: Running three models (especially large ones) in sequence is resource-intensive. Depth Anything v2 (with ViT-L) and SAM (with ViT-H) are huge. Qwen-VL is also huge. This is fine for an offline analysis or powerful machine, but not for an embedded device as is. We tackled this by exploring smaller variants: Depth Anything ViT-B still did well and was faster. SAM has ViT-B and ViT-L versions that are faster (with some accuracy trade-off on fine details). Qwen-VL unfortunately doesn’t have a “small” version publicly; it’s by nature large. But one could use a smaller multimodal model (like MiniGPT4 or BLIP2 with a 2B model) to sacrifice some language quality for speed. Another tactic is to run models on different hardware in parallel (if available). In an autonomous vehicle, perhaps a dedicated hardware accelerator could run the depth model (or the car might already have a depth estimation from LiDAR or stereo – if so, we might not need Depth Anything at all, just use actual sensor depth to create a DAM). The segmentation could also be accelerated with dedicated NPUs since it’s basically a lot of matrix ops. There is also a question of memory: storing and transferring the fused images. That overhead is negligible compared to running these nets though. So main cost is compute. As pointed out, Depth Anything v2 can reach 38 FPS on a V100 for decent resolution, meaning if optimized, it’s not a bottleneck. SAM’s default is slower, but EfficientSAM promised a 20x speed-up (which could give maybe 10 FPS instead of 0.5 FPS on a CPU for segmentation). So by selecting the right variants, real-time is within reach. We explicitly consider this in future work on video.

Multimodal Prompt Engineering: Another challenge was how to phrase the prompt to Qwen-VL effectively. Initially we tried a question format (“What does the depth and segmentation show?”). The answers were okay but not very detailed. We found that using an imperative “Write a paragraph describing...” yields a more comprehensive answer. This is consistent with how instruction-tuned models often respond more verbosely to an explicit instruction to “describe” rather than a short question. However, for direct QA like “Which object is closest?”, a question format is appropriate. Qwen-VL in our tests answered these correctly (one-sentence answers). But if our goal is to have a chatty explainer, the paragraph style works. Therefore, one needs to adjust the prompt depending on desired output length and content. We embedded figure references in the prompt which rely on Qwen’s training to understand (likely from academic papers or webpages it saw). Many language models are not explicitly trained to handle “Figure 1 is X, Figure 2 is Y” but Qwen-VL apparently learned from data that that format means multiple image references. It was interesting to see it handled that without special finetuning from our side. A challenge is if we had more than maybe 2-3 images, the prompt would get longer and might confuse the model’s token context. We used at most 3, which was fine. Also, we had to be careful not to cause Qwen to think it should output something not desired – for example, we started our prompt with “Figure 1 is a depth map...” to ground it. If we started with just “Depth map: [some info]. Segmentation map: [some info]. Describe the scene.”, there’s a chance it might misunderstand as if those were part of an input sequence to analyze. But it likely would still parse it. The point is, multimodal prompt design is a new area with not many established best practices, so it took trial and error. We considered using a structured prompt like: “*i*Vision₁... image1... *i*/Vision₁*i*Vision₂... image2... *i*/Vision₂*i*Text₁Write a paragraph using both images₁/Text₁” because some models (like OFA or PaLI) use such tagging, but Qwen-VL didn’t require that explicit markup from what we gather.

Domain Adaptation: Though Depth Anything was trained to be robust across domains and we even added an adversarial alignment, any model can face issues if the input domain is very different (e.g., a nighttime infrared camera image). Similarly, SAM is trained on diverse images but maybe not thermal images or very low-light ones. Qwen-VL’s vision encoder too might not handle weird sensor modalities. In autonomous driving, multiple sensor types could be fused. Our pipeline currently assumes a normal RGB camera image input. If given a different modality, Depth Anything might fail entirely to predict meaningful depth (for example, if given a LiDAR projected depth image, ironically, the depth model doesn’t need to exist then because you already have depth!). We’d rather incorporate LiDAR directly if available. For segmentation, if the camera is a different spectrum (infrared), SAM

might not work because it was trained on RGB. Solutions for domain adaptation: fine-tune or calibrate each component on the new domain. Depth Anything’s design does help bridging synthetic and real as shown by the domain adaptation phase (which improved cross-dataset performance significantly). If we moved to, say, a fisheye camera, we might need to adjust the model or preprocess to undistort images before feeding. We did not address specific domain shifts in our experiment (KITTI and others were similar perspective color images). But it’s a consideration for deployment. Also, if the autonomous system uses multiple cameras (front, side, etc.), our system could be applied per camera or even combined if Qwen can handle multi-image input (like a surround view). That would be an extension (like “Figure 1: front view depth. . . Figure 2: side view depth. . .” etc – it might be able to integrate multi-view info too). But we’d need to be careful that Qwen doesn’t get confused or overload context.

Memory and State for Interactive QA: Currently, each question is answered independently given the image. For interactive dialogue, the model would need to maintain context of previous questions or focus on certain objects persistently. Qwen-VL-Chat is a version meant for multi-turn dialogue, which we could use to allow follow-up questions. One challenge is feeding the image again or referencing it in follow-ups. Likely, one would keep the image embeddings in memory and just keep the dialogue going, with the user and assistant turns. Qwen-VL-Chat presumably does that (they mention it can hold multi-turn multi-modal conversations). We haven’t tested multi-turn thoroughly, but we anticipate needing to prompt it carefully like: user: “(image). . . question1”, assistant: answer1; user: “question2 (with reference like ‘that car’?)”, assistant: should remember ‘car’ from image. Standard vision-language chat does handle that to some extent by reusing the image context. A technical challenge is that if the chat session is long, you can’t feed the image every time due to context length. Models like GPT-4 have some way of storing image features across turns (maybe by internal state). For open-source, maybe not. Qwen-VL-Chat being specifically built might handle it. So to incorporate that in our system, we might either use Qwen-VL-Chat directly (if released) or emulate it by keeping track of which segment corresponds to what and re-embedding references. We did not implement multi-turn in this project due to time, but the plan is to allow a conversation like: “User: What do you see? Assistant: I see a car and building. User: How far is the car? Assistant: 20 m away. User: Highlight the car. Assistant: (could output a marked image or describe location).” Each of those requires some handling. Possibly mixing some image generation (like drawing on image to highlight, which Qwen-VL might not directly output because it outputs text). We might instruct Qwen to output coordinates to highlight, but that’s beyond current scope.

In addressing these challenges, we realized the benefit of the system’s modular nature. Each part can be improved or replaced without overhauling everything. For example, if a better segmentation model comes out (maybe one that also outputs labels, like a panoptic segmentation model), we can drop SAM for that. If a future Qwen-VL version can inherently interpret depth maps numerically, we won’t need workarounds for numeric output. The engineering solution of using intermediate files (images) to communicate between models proved robust – even though it’s somewhat brute-force (visualizing internal data for the LLM to read), it uses each model in its comfort zone (vision models produce images, language model consumes images and text). We also leveraged existing tools like ONNX runtime to handle performance issues for SAM.

In conclusion, while our system currently has limitations (notably the absence of explicit numeric distance in answers and the reliance on the language model’s understanding to label segments), we have charted a path to mitigate these. Adding an explicit depth-to-text step and semantic labeling step are high priorities. Many of the challenges (like segmentation labeling and real-time constraints) can be addressed by incorporating additional subsystems or optimizations, which we discuss in the next section on future work. The challenges we solved (like fusing the modalities in prompts and extracting the DAM) provide a foundation for those next steps. The key insight is that such a combination of models is feasible and effective, but it requires careful handling of the interfaces between them – something we tackled through prompt design and data conversion (e.g., attention maps, colored masks). This ensures that the strengths of one model (e.g., precise depth, broad segmentation) directly compensate for weaknesses of another (LLM’s lack of explicit structure or the vision model’s lack of explanation).

6 Future Work

The promising results from our multimodal QA system open up several avenues for further development. In this section, we outline key areas of future work to enhance the system’s capabilities, performance, and applicability, especially with an eye towards real-time and interactive use in autonomous vehicles.

Real-Time Video Processing: One immediate extension is to move from single images to continuous video input. Processing video in real-time would enable the system to describe dynamic events (e.g., “a pedestrian is crossing the street in front of the car”) and maintain context over time. Achieving this entails optimizing each component and introducing temporal consistency mechanisms:

- *Temporal Consistency for Depth (DAM smoothing):* As Depth Anything v2 already includes a spatiotemporal

fusion module (STFM) for adjacent frames, we can utilize that to get more stable depth and attention maps between frames. The Depth Attention Map (DAM) could be averaged or low-pass filtered over a short window to reduce flicker (so that slight changes in attention between frames don't confuse the language model with inconsistent prompts). Ensuring the DAM evolves smoothly will help Qwen-VL focus on changes that truly matter (like a new object appearing) rather than momentary shifts in attention.

- *Object Tracking in Segmentation:* We plan to integrate an object tracking module to assign consistent IDs to segments across frames. For example, if SAM (or another segmentation model) outputs masks per frame, a tracker (like ByteTrack or SORT) can link those masks over time. This way, if the user asks “Where did the blue car go now?”, the system knows which segment corresponded to the blue car in previous frames and can find it in the current frame. A stable segmentation also aids Qwen-VL in maintaining context (possibly by referencing the same object via the same label). With tracking, we could refer to “the car” consistently rather than “purple region” changing color each frame.
- *Frame Selection:* Running the full pipeline on every video frame (e.g., 30 FPS) might be overkill and computationally heavy. We can adopt a strategy to select key frames or events: for instance, run the full analysis at a lower frame rate (say 5 FPS) and interpolate in between, or trigger the pipeline when something changes (like a new object appears or an object significantly changes depth/distance). The Depth Anything model's confidence and attention could themselves signal when a change happens (if an object moves closer rapidly, the attention might spike there, indicating an event).
- *Performance optimization:* As discussed earlier, employing smaller model variants or pruning can help meet real-time constraints. Additionally, on hardware like automotive-grade SoCs, using quantized models or accelerator cores (TPUs/NPUs) will be important. We may explore distilling Qwen-VL's knowledge into a smaller model for faster response, though that is a research challenge on its own.

Interactive Question Answering and Dialogue: We aim to transform the system from a one-shot QA to an interactive dialogue agent for scene understanding:

- *Qwen-VL-Chat integration:* If available, we would integrate the chat-specific version of Qwen-VL, which can handle multi-turn dialogues and follow-ups. This model would maintain the conversation history so that the user can ask sequential questions without re-providing the image each time. We will need to ensure the image (or its features) remains in context. Possibly the initial turn would include the image and subsequent turns rely on the model's memory of it (Qwen-VL-Chat likely has a mechanism for that).
- *Referring expressions:* A user might point at or refer to an object by some descriptor given in an earlier answer (e.g., “that purple region” or “the first car”). The system should resolve this reference to a specific object/segment. We can maintain a mapping from descriptors to segments (since we know which segment was purple, etc.). We could even assign labels like “Object A, Object B” in the conversation explicitly. In responses, we might say “Object A (a car) is ...; Object B (a building) is ...” and then the user can ask about Object A by name. This is one approach to make referencing easier. Another is to use a visual interface where the user can click on an object in the segmentation map to ask about it. This would be outside the pure language domain, but it's very practical for an interactive tool (like “What is this?” with a click).
- *Extended Knowledge and Reasoning:* At present, Qwen-VL describes what it sees. In future, we might incorporate background knowledge, e.g., if the user asks “Is that car likely an emergency vehicle?”, the model might not know just from the image unless it recognizes specific features (like sirens or markings). If Qwen were connected to a knowledge base or had been trained on contextual knowledge (e.g., common knowledge that “white and black cars with certain patterns are police cars”), it might infer more. GPT-4 style models can sometimes do that. Future versions of Qwen or other LLMs might incorporate more world knowledge. We could also feed external information into the prompt (like “Note: vehicles with a certain marking in this region are police cars”) if needed.
- *Handling Uncertainty and Errors in Dialogue:* Interactive settings require the system to handle cases where it's not confident. Qwen-VL already sometimes hedges (“it seems like... but not certain”). In a safety-critical domain like driving, we might want the system to explicitly say when it doesn't know (“I'm not sure what that object is, it might be ...”) rather than hallucinating. Fine-tuning Qwen on a style that expresses uncertainty for ambiguous visual inputs would be valuable. Alternatively, use the confidence outputs from Depth Anything and from any classification model to decide if the model should answer or defer. The system could respond, “I'm not confident enough to answer that.”, which is better than a wrong guess if a human's relying on it.

Embedding Numerical Depth and Spatial Metrics: We plan to enrich the prompts or the model outputs with actual quantitative data:

- *Depth Value Embedding:* As discussed, we can attach actual depth readings for specific segments. A straightforward method is: for each prominent segment (say each SAM mask that corresponds to an object), compute the average or median depth from Depth Anything's output, convert it to a distance (with camera calibration known, Depth Anything might output disparity or relative depth but presumably can be scaled to metric if trained on KITTI with LiDAR). Then, either annotate the segmentation image with that number (like put text "15m" near the object) or append a line in the prompt text "(The car in purple is 15 m away.)". We will experiment with these approaches to see which yields better incorporation by Qwen. Putting it as text might directly make Qwen mention it in answers. Putting it in the image might allow Qwen's OCR to pick it up as part of vision input (since Qwen has text-reading ability as per Qwen-VL paper). There's a bit of a hackiness in expecting the model to OCR numbers, but it's doable. Alternatively, a future large model might allow structured input (like a table of numbers or an API call for calculation), but that's beyond current usage.
- *Spatial Coordinates and Relationships:* We could also embed approximate spatial coordinates of objects (e.g., car is at bearing 5° left, 20m distance). This could be especially useful if the QA gets into navigation or positioning. For instance, a user might ask "Where is the car relative to our lane?". If we had a map of lane lines and car position in them, we could answer robustly. Depth + segmentation could let us infer if the car is in our lane vs adjacent (by overlapping segmentation with known lane segmentation). These are additional visual processing tasks (lane detection, free space estimation) that can be integrated. Depth Anything doesn't directly give lanes, but segmentation could (with training for road markings). In future, adding a specialized model for lane or drivable area segmentation and including that as another visual prompt could allow questions like "Can I pass this car?" (the model could infer if there's space or if it's one lane each direction).
- *Proactive Warnings and Explanations:* With numeric depth, the system could also take on an advisory role: e.g., "The vehicle ahead is very close (10 m), you should maintain more distance." This goes slightly beyond QA into driving assistance. While not asked, a system might proactively alert if a question implies a safety concern ("Is that pedestrian going to cross?" might prompt the system to respond carefully with a warning if uncertain).

Scaling to More Complex Scenes and Datasets: We intend to test and adapt the system to more challenging environments:

- *Different Weather/Lighting:* Rainy, night, or foggy scenes could degrade segmentation and depth. We might incorporate models specialized for those conditions or retrain Depth Anything with augmentation for rain/fog (some research uses domain adaptation for weather).
- *Datasets for Evaluation:* We might formally evaluate on datasets like the Waymo Open dataset or Argoverse for describing driving scenes, if such ground truth captions or descriptions existed. There's no standard "driving scene QA" dataset, but we could create one. For example, using the nuScenes dataset (which has 3D annotated bounding boxes), we could generate questions like "How many cars are visible? Is any car closer than 10m? What is the distance to the nearest pedestrian?" and use the ground truth as the answer. Then test how well the system answers. Automating that would give a quantitative benchmark for the integrated system. This could guide improvements and measure progress (as a new task evaluation).
- *Benchmarking the Language Model's Visual Prompt Understanding:* As we add more inputs (like original image, depth, segmentation, lane markings), it becomes a complex multimodal prompt. We might reach the limit of what a single model can digest in one go. If so, future research might use a multi-step reasoning pipeline (like first ask the model to analyze depth, then segmentation, then combine). Tools like program-aided LLM reasoning (e.g., using a small script that the LLM can execute to combine info) might be beneficial. For instance, one could prompt the LLM to output a short JSON of key numbers (like distances of objects), then do calculations, then have it produce the paragraph. This "chain-of-thought" or structured approach can increase reliability. It's part of future exploration: making the LLM not a black box but a controllable reasoner.

Integration of Additional Modalities: An autonomous vehicle often has more than just a forward camera. Future expansions include:

- *LiDAR and Radar*: If LiDAR is available, one could bypass Depth Anything for primary depth (since LiDAR gives precise distances). But Depth Anything could still be useful to fill in gaps (LiDAR is sparse in some areas) or to distill LiDAR data into an attention map. Alternatively, just compute an attention map directly from LiDAR point cloud (like which points are moving or closest). That might be simpler. Similarly, radar gives velocity and rough position of objects, which could answer questions about moving speed (“Is that car speeding?”). We could incorporate radar data by textual or visual encoding (like an arrow or number indicating velocity on the object).
- *Audio*: Not directly relevant for visual QA, but if the scenario included an audio feed (sirens, horns), an interactive system could mention them. E.g., if a siren is heard, the system might say “I hear an ambulance siren” which combined with vision (an ambulance segment) gives a fuller picture. This requires an audio analysis model integrated similarly (transcribing audio events to text or icons).
- *Map/GPS*: Knowing the vehicle’s location or route could allow questions like “How far to destination?” which goes beyond the immediate scene. That enters navigation QA territory. It might be out-of-scope, but an advanced assistant could incorporate that. For scene-centric, not needed.

User Interface and User Experience: Outside of technical performance, how the user interacts with this system in a vehicle matters:

- *Augmented Reality (AR) Display*: Instead of just a voice or text answer, the system could highlight objects on a heads-up display (HUD) or tablet. For instance, if the user asks “Where is the pedestrian you mentioned?”, the system could outline the pedestrian on the video feed. Achieving this means linking the object mentioned in text to the visual, which we have via segmentation masks and tracking. We could then overlay that mask on a screen. Qwen-VL itself won’t output an image, but our system could easily retrieve the mask from SAM and project it. This leads to a rich interactive experience: the user asks, the system answers and visually points out.
- *Voice interaction*: In a car, voice is natural. Qwen-VL can generate text; text-to-speech can convert it to voice. For input, speech-to-text can feed the question to the system. Care must be taken that technical terms (like “Figure 1”) are not needed from user obviously. They would just say “How far is that car?” – the system would need to know “that car” refers to, say, the one in front. Possibly by default we assume the nearest car if not specified. Or the user could say “the blue car to the left” and the system has to map that description to a segment (which it can, since segmentation has color and position).
- *Safety Considerations*: If deployed in a vehicle, we must ensure the system’s advice doesn’t encourage unsafe behavior. This might involve a safety filter or additional logic. For example, if asked “Can I run the yellow light?”, the system should not simply say “Yes, you can make it” even if visually it looks possible, because it should incorporate traffic rules. This might be beyond pure perception QA and into ethical/policy programming. For our research prototype, we focus on descriptive and factual QAs, but a real system might also need to decline certain requests (like those that go beyond just describing into advising potentially illegal actions).
- *Domain Adaptation to user language*: Drivers might use slang or different phrasing. Fine-tuning Qwen-VL on a corpus of in-car commands or questions might improve it. It’s already multilingual which helps if the user isn’t English-speaking.

Retraining and Domain Adaptation of Models: As newer data comes (e.g., new types of vehicles, unusual objects on road like scooters, delivery robots), it might be useful to fine-tune SAM (or replace it with SAM-CP fine-tuned on a driving dataset for better vehicle/person masks) and fine-tune Qwen on domain-specific QAs. The modular design means we can iterate on individual components as new research emerges (e.g., if a future “SAM 2.0” provides better segmentation with labels, we can integrate that to directly provide those labels to Qwen).

Explainability and Debugging: One interesting future aspect is to have the system not only answer questions but explain its reasoning. For instance, Qwen-VL’s answer about depth is essentially an explanation: “because the depth map is lighter for this object, it’s closer.” We could push this further by asking the model to justify its answers in a step-by-step manner. This would be useful to trust the system or debug it. Large language models can output a reasoning chain if prompted (e.g., using few-shot examples of reasoning). We might instruct it: “Answer the question and then explain how you used the depth and segmentation to get the answer.” This could produce output like: “The nearest object is the car. I determined this because on the depth map the car’s region is brightest, meaning it’s closest. The segmentation also isolates the car as a distinct object. Therefore, the car is closer than the rest.” Such explanations are valuable in safety-critical settings to verify the model isn’t making something up.

Generalization to Other Domains: Although our focus is road vehicles, the concept of combining spatial maps with language models is broadly applicable. In the future, we could apply a similar pipeline to robotic manipulation (depth + segmentation of a table-top scene feed into an LLM to instruct a robot), or medical imaging (though that's 2D slices rather than depth maps, but conceptually similar segmentation + maybe attention heatmaps of anomalies to explain a diagnosis). The successful principles here could guide those applications.

In summary, the future work is geared towards making the system more robust, more informative (with numbers and labels), and more user-friendly (real-time interactive). Particularly for autonomous driving, moving to a continuous analysis that can narrate or answer queries on the fly is an exciting direction. We anticipate that addressing these items will move this system from a proof-of-concept to a practical co-pilot intelligence that can enhance situational awareness and safety.

7 Conclusion

In this report, we presented a comprehensive vision-language system that fuses depth-based attention and segmentation cues to enrich a language model's understanding of road scenes. Our approach demonstrates that large multimodal models like Qwen-VL can be guided with structured visual prompts to produce more spatially aware and interpretable outputs. By integrating the Depth Attention Map (DAM) from Depth Anything v2 and the Segment Anything Model (SAM) masks as intermediate representations, we provide the language model with an interpretable "focus map" of where important objects and depth transitions occur. This leads to more detailed and grounded descriptions — for instance, correctly identifying which objects are closer, how they are separated in the scene, and describing the scene layout with reference to those visual cues.

We thoroughly discussed the methodologies behind the core components: Depth Anything's hierarchical transformer-CNN architecture with cross-modal feature alignment and advanced training losses that yield state-of-the-art monocular depth accuracy, and SAM's ViT-based image encoder with prompt-conditioned mask decoding that produces high-quality segmentation masks. We elaborated on how each model's design (attention mechanisms, loss functions like focal/Dice for SAM and photometric/geometry for Depth Anything) contributes to their performance and how we leveraged their outputs. Notably, we exploited the internal attention weights of the depth model to create the DAM, a novel use of the model's reasoning process as an explicit input to another AI component.

The fused system was evaluated on its ability to address vehicle distance estimation and general scene understanding. Depth Anything v2 demonstrated excellent performance on KITTI and related benchmarks, indicating reliable depth estimates (AbsRel 0.076 on KITTI, significantly better than prior work). SAM similarly provides strong segmentation priors, capturing objects even without training on the specific scene context, although lacking labels. With these, our Qwen-VL based QA system could answer questions like "Which object is closest?" or "Is the pedestrian in front of the car?" with an appropriate understanding of the spatial context, something that end-to-end vision-language models without such structured prompts might struggle with. We showed qualitative examples where the model accurately describes a complex scene (distinguishing foreground and background elements and their depth relation) and reasoned about occlusion and relative distance. This indicates an advancement in the interpretability and reliability of AI-generated scene descriptions.

We also discussed challenges, notably Qwen-VL's current inability to output numeric distances or identify unlabeled segments, and proposed solutions like augmenting prompts with explicit depth values and integrating lightweight classifiers for segment labeling. These enhancements, along with extending the system to handle temporal sequences and interactive dialogue, form the basis of our future work plan.

The system's modular and interpretable design is a key strength. Each module (depth, segmentation, language) contributes a human-understandable piece: depth gives a heatmap, segmentation gives colored regions, and the language model gives a fluent explanation. This modularity not only made development and debugging more tractable (we could improve one part without retraining the others) but also provides multiple points of verification. For example, one can inspect the DAM and segmentation output to trust the final answer — a step towards more transparent AI in safety-critical domains like autonomous driving. Moreover, by maintaining these intermediate representations, we create opportunities for incorporating human feedback; a user or engineer could adjust the segmentation or highlight a region of the depth map and ask the system to reconsider, thereby fine-tuning the response without retraining the model.

In conclusion, our work demonstrates a pathway to empower large language models with a richer understanding of visual scenes by injecting structured visual knowledge into their input. For the domain of road vehicles, this means an AI assistant that can not only caption a dashcam image, but also analyze the 3D layout, estimate distances, and ground its responses in the physical context — capabilities crucial for explaining and reasoning in autonomous driving. The improvements outlined for future work will further push the system towards real-time, interactive operation and greater accuracy, with the ultimate goal of providing an intelligent co-pilot that can

answer questions like a knowledgeable human observer, but with the enhanced precision and multi-modal insight of AI.

By bridging cutting-edge depth estimation, segmentation, and language modeling, we have taken a significant step toward holistic scene understanding. This integrated approach paves the way for safer and more transparent autonomy, where an AI can not only perceive the world but also communicate its understanding effectively to humans. We expect that the techniques and insights from this project will generalize beyond driving scenarios to any application where explaining complex visual data is required — embodying a move towards AI systems that are not just accurate, but also interpretable and collaborative.

References

- [1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [2] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, et al. Flamingo: a visual language model for few-shot learning. *Advances in neural information processing systems*, 35:23716–23736, 2022.
- [3] S. Bai, K. Chen, X. Liu, J. Wang, W. Ge, S. Song, K. Dang, P. Wang, S. Wang, J. Tang, et al. Qwen2. 5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025.
- [4] S. F. Bhat, I. Alhashim, and P. Wonka. Adabins: Depth estimation using adaptive bins. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4009–4018, 2021.
- [5] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [6] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille. Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs. *IEEE transactions on pattern analysis and machine intelligence*, 40(4):834–848, 2017.
- [7] P. Chen, L. Xie, X. Huo, X. Yu, X. Zhang, Y. Sun, Z. Han, and Q. Tian. Sam-cp: Marrying sam with composable prompts for versatile segmentation. *arXiv preprint arXiv:2407.16682*, 2024.
- [8] B. Cheng, A. Choudhuri, I. Misra, A. Kirillov, R. Girdhar, and A. G. Schwing. Mask2former for video instance segmentation. *arXiv preprint arXiv:2112.10764*, 2021.
- [9] Y. Duan, X. Guo, and Z. Zhu. Diffusiondepth: Diffusion denoising approach for monocular depth estimation. In *European Conference on Computer Vision*, pages 432–449. Springer, 2024.
- [10] S. Erisen. Sernet-former: semantic segmentation by efficient residual network with attention-boosting gates and attention-fusion networks. *arXiv preprint arXiv:2401.15741*, 2024.
- [11] Z. Gao, Z. Tong, L. Wang, and M. Z. Shou. Sparseformer: Sparse visual recognition via limited latent tokens. *arXiv preprint arXiv:2304.03768*, 2023.
- [12] E. Ghysels, A. Sinko, and R. Valkanov. Midas regressions: Further results and new directions. *Econometric reviews*, 26(1):53–90, 2007.
- [13] M. S. Hamid, N. Abd Manap, R. A. Hamzah, and A. F. Kadmin. Stereo matching algorithm based on deep learning: A survey. *Journal of King Saud University-Computer and Information Sciences*, 34(5):1663–1673, 2022.
- [14] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, et al. Segment anything. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4015–4026, 2023.
- [15] J. Li, D. Li, S. Savarese, and S. Hoi. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International conference on machine learning*, pages 19730–19742. PMLR, 2023.

- [16] J. Li, D. Li, C. Xiong, and S. Hoi. Blip: Bootstrapping language-image pre-training for unified vision-language understanding and generation. In *International conference on machine learning*, pages 12888–12900. PMLR, 2022.
- [17] H. Liu, C. Li, Q. Wu, and Y. J. Lee. Visual instruction tuning. *Advances in neural information processing systems*, 36:34892–34916, 2023.
- [18] O. Özyeşil, V. Voroninski, R. Basri, and A. Singer. A survey of structure from motion*. *Acta Numerica*, 26:305–364, 2017.
- [19] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR, 2021.
- [20] R. Ranftl, A. Bochkovskiy, and V. Koltun. Vision transformers for dense prediction. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 12179–12188, 2021.
- [21] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.
- [22] O. Ronneberger, P. Fischer, and T. Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical image computing and computer-assisted intervention—MICCAI 2015: 18th international conference, Munich, Germany, October 5–9, 2015, proceedings, part III 18*, pages 234–241. Springer, 2015.
- [23] J. L. Schonberger and J.-M. Frahm. Structure-from-motion revisited. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4104–4113, 2016.
- [24] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [25] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [26] Q. Wan, Z. Huang, B. Kang, J. Feng, and L. Zhang. Harnessing diffusion models for visual perception with meta prompts. *arXiv preprint arXiv:2312.14733*, 2023.
- [27] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo. Segformer: Simple and efficient design for semantic segmentation with transformers. *Advances in neural information processing systems*, 34:12077–12090, 2021.
- [28] Y. Xiong, B. Varadarajan, L. Wu, X. Xiang, F. Xiao, C. Zhu, X. Dai, D. Wang, F. Sun, F. Iandola, et al. Efficientsam: Leveraged masked image pretraining for efficient segment anything. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16111–16121, 2024.
- [29] C. Yu, B. Xiao, C. Gao, L. Yuan, L. Zhang, N. Sang, and J. Wang. Lite-hrnet: A lightweight high-resolution network. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10440–10450, 2021.